

Software Supply Chain Attacks and Build Provenance

A Paper-Style Report on npm Ecosystem Risks and Provenance-Based Defense

NG, Chun Sing (r14922186) OU, Chia-Yun (r14922072)

May 31, 2026

Abstract

Modern software development heavily depends on open-source packages, package managers, version control platforms, continuous integration and continuous deployment (CI/CD) pipelines, and third-party package registries. These mechanisms make software reuse and automated release processes more convenient, but they also place the development workflow on top of complex trust relationships. When developers run commands such as `npm install`, `pip install`, `mvn install`, or `conda install`, they are not merely downloading code. They are also implicitly trusting a large number of transitive dependencies, maintainer accounts, registry metadata, installation scripts, build workflows, and release pipelines.

This report examines software supply chain attacks and the role of build provenance in defending against such attacks. It first introduces the definition of software supply chain security, explains why the risk has increased, and discusses why OWASP Top 10:2025 ranks Software Supply Chain Failures as A03. It then analyzes two npm ecosystem incidents: Axios and TanStack. The Axios incident shows how stolen maintainer credentials, malicious package publication, hidden dependencies, and `postinstall` scripts can introduce malware through an ordinary installation process. The TanStack incident further demonstrates that even when an artifact has valid build provenance, provenance may not prevent malicious publication if a trusted CI/CD workflow or cache trust boundary has already been compromised.

The central argument of this report is that build provenance is necessary, but not sufficient as a complete defense. Provenance improves

artifact traceability by making the relationship among source commits, builder identities, build workflows, and artifact hashes more verifiable. However, it does not prove that an artifact is benign, nor does it guarantee that the build environment was not compromised. Effective software supply chain defense therefore requires a layered strategy that combines dependency scanning, lockfile discipline, install-time verification, maintainer account protection, CI/CD isolation, cache hardening, credential restriction, signed attestations, and provenance verification.

Keywords: software supply chain security, build provenance, SLSA, Sigstore, npm, CI/CD security, dependency attack

Table of Contents

1 Introduction	5
2 Background: Software Supply Chain Security	6
2.1 Definition of Software Supply Chain	6
2.2 Why Software Supply Chain Failure Became a Top Risk	6
2.3 Economic and Ecosystem Impact	7
3 Threat Model and Attack Surfaces	8
4 Case Study I: Axios npm Compromise	8
4.1 Incident Overview	8
4.2 Attack Flow	9
4.3 Limitations of Common Defenses	9
5 The Core Gap: No Tamper-Evident Chain	10
6 Build Provenance as a Defense Mechanism	11
6.1 Concept of Build Provenance	11
6.2 What Provenance Can Prove	11
6.3 Verification: Turning the Record into Prevention	12
6.4 Technology Stack and Workflow	13
6.5 Adoption Status	13
7 Case Study II: TanStack npm Supply-Chain Compromise	15
7.1 Incident Overview	15
7.2 How the Attack Worked	15
7.3 Why This Challenges Provenance	16
7.4 Industry Response: OpenAI	16
8 Toward a Layered Defense Strategy	17
8.1 Dependency Layer	17
8.2 Source Layer	17
8.3 Build Layer	17
8.4 Release Layer	18
9 Discussion	18
9.1 Practical Value of Build Provenance	18
9.2 Fundamental Limitation of Build Provenance	19
9.3 Synthesis of the Two Case Studies	19

10 Conclusion

20

1 Introduction

Modern software development is no longer a closed process fully controlled by a single development team. Developers routinely obtain existing libraries through package managers, while build, test, release, and deployment tasks are increasingly delegated to automated tools. Common commands in everyday development include the following:

```
npm install    pip install    mvn install    conda install
```

From an engineering perspective, these commands improve productivity because teams can quickly reuse existing software components. However, this convenience also expands the attack surface. When a direct dependency is installed, the package manager may also resolve and download dozens of transitive packages. In some ecosystems, the installation process may also execute lifecycle scripts such as `preinstall`, `install`, or `postinstall`. In other words, a developer may appear to be running a single installation command, while in practice trusting many upstream components and intermediate processes that are not manually inspected.

The danger of software supply chain attacks arises from this structure of implicit trust. An attacker does not necessarily need to compromise the final application directly. Instead, the attacker can target any trusted element in the chain, such as a dependency package, maintainer account, package registry, build cache, CI/CD workflow, or release pipeline. Once a trusted upstream element is compromised, malicious code can enter many downstream projects through normal installation or release workflows.

This report focuses on the following research question: **Can build provenance effectively prevent software supply chain attacks?** The position of this report is deliberately cautious. Build provenance can make the origin, build process, and release record of an artifact more traceable, and it provides a foundation for install-time verification. However, if a legitimate workflow itself is abused by an attacker, provenance may still produce a record that is correct but incomplete. Therefore, this report treats provenance as a key mechanism within defense in depth, rather than as a complete standalone solution.

2 Background: Software Supply Chain Security

2.1 Definition of Software Supply Chain

According to NIST, a software supply chain is a set of steps that create, transform, and assess the quality and policy conformance of software artifacts. In practice, this supply chain includes source code, third-party dependencies, version control systems, build tools, CI/CD pipelines, package registries, artifact repositories, release workflows, and deployment mechanisms. Therefore, the security of a modern software product depends not only on the code written by the development team, but also on the entire environment through which the software is developed, built, and released.

A software supply chain attack occurs when an attacker compromises or manipulates a trusted node in this chain. The compromised node may be a dependency package, maintainer account, build cache, CI/CD workflow, registry account, signing process, or release pipeline. The attacker's goal is not simply to exploit a vulnerability inside the final application. Instead, the attacker abuses the existing trust relationships among developers, tools, packages, and registries. Because popular packages are often reused by many projects, a single upstream compromise can create large downstream impact.

2.2 Why Software Supply Chain Failure Became a Top Risk

OWASP Top 10:2025 ranks *Software Supply Chain Failures* as A03, which reflects a shift in the risk landscape of application security. Traditional application security often focuses on internal defects in applications, such as injection, authentication failure, insecure design, or access control failure. By contrast, software supply chain security focuses on how an application is built, which external components it depends on, who publishes those components, and whether the final artifact can be verified.

OWASP Top 10:2025 is not determined directly by a single dataset; rather, it is data-informed. Its ranking combines contributed testing data, CVE-based exploit and impact scores, and community survey results. For A03:2025 Software Supply Chain Failures, OWASP reports a 5.72% average incidence rate, 215,248 total occurrences, and 11 mapped CVEs. These

numbers should be interpreted as the risk signals observable through current testing data and tools, not as the global frequency of real-world attacks. In fact, supply chain attacks often involve newly published malicious packages, credential abuse, or manipulated CI/CD workflows, which may not immediately appear in CVE databases or traditional scanning results. As a result, the real-world risk may be underrepresented.

The problem has become highly significant because of several structural factors. First, modern applications often contain large dependency graphs, where a small number of direct dependencies can introduce many transitive dependencies. Second, package managers are highly automated: they resolve versions, download packages, read metadata, and sometimes execute installation scripts. Third, CI/CD systems have become central to build and release processes, making workflow permissions, runner environments, OIDC tokens, and build caches important attack surfaces. Fourth, developers rarely inspect every package version, tarball, build log, artifact hash, or provenance record manually. These factors allow an attacker to influence many downstream systems by compromising one trusted node.

2.3 Economic and Ecosystem Impact

The financial impact of supply chain attacks is difficult to measure completely because many incidents do not publicly disclose detailed losses. Nevertheless, breach-cost data indicates that supply chain compromise is expensive. IBM 2025 breach-cost data reports that third-party vendor and supply chain compromise averaged USD 4.91M per attack and was identified as the second-most prevalent and second-costliest data breach vector. Such costs may include incident response, downtime, credential rotation, CI/CD rebuild, legal risk, and reputational damage.

At the same time, open-source package risk continues to increase. The ReversingLabs 2026 software supply chain security report states that malicious open-source package detections increased by 73% in 2025, with npm accounting for nearly 90% of detected open-source software malware. This trend suggests that attackers view package ecosystems as a high-return attack surface. The npm ecosystem is particularly important because of its large package scale, fast release rhythm, automated installation workflow, and highly transitive dependency structure.

3 Threat Model and Attack Surfaces

Software supply chain attacks can be analyzed across four layers: the dependency layer, the source layer, the build layer, and the release layer. Each layer contains different attack surfaces and requires different types of security controls.

Table 1: Major attack surfaces in the software supply chain

Layer	Possible Attack Surface
Dependency	Malicious packages, typosquatting, dependency confusion, transitive dependency compromise, and abuse of <code>preinstall</code> , <code>install</code> , or <code>postinstall</code> lifecycle scripts.
Source	Compromised maintainer accounts, malicious commits, review bypass, weak protected branch settings, and overly broad release permissions.
Build	CI/CD workflow tampering, build cache poisoning, token extraction from runner memory, unsafe fork pull request workflows, and overprivileged OIDC tokens.
Release	Registry account compromise, artifact substitution, missing signatures, missing attestations, and missing provenance verification.

In the npm ecosystem, lifecycle scripts are an especially important attack surface. Scripts such as `preinstall`, `install`, and `postinstall` may execute automatically during package installation. Therefore, if an attacker can publish a package version containing a malicious lifecycle script, the victim may execute malicious code before the application even imports the package. This makes the installation phase itself an important execution vector for supply chain attacks.

4 Case Study I: Axios npm Compromise

4.1 Incident Overview

Axios is a widely used HTTP client in the JavaScript ecosystem. According to the presentation material, Axios has approximately 100 million weekly downloads. On March 31, 2026, Axios suffered an npm supply chain compromise. The attacker used stolen maintainer credentials to publish malicious package versions, with an exposure window of approximately three hours. The malicious version injected an attacker-controlled hidden depen-

dency and used a `postinstall` script to install a Remote Access Trojan (RAT) during the normal installation process.

This incident is representative because the attack did not require the user to actively run the application. Once the user executed `npm install axios`, the installation process itself could trigger the malicious script. In other words, the attack occurred during the development and installation phase rather than after deployment at runtime. This also makes the initial compromise path harder for traditional application-level security controls to observe.

According to the presentation material, this incident also affected OpenAI's macOS app-signing pipeline, putting code-signing certificates for ChatGPT Desktop and Codex at risk. This point highlights the amplification effect of supply chain attacks. If the affected environment is not merely an end-user machine but a development environment with signing credentials, build privileges, or release permissions, the consequences can spread further to downstream users.

4.2 Attack Flow

The Axios incident can be conceptualized as the following attack flow:

```
npm install axios
  -> malicious axios version
    -> attacker-controlled hidden dependency
      -> postinstall script executes
        -> Remote Access Trojan installed silently
```

The danger of this flow is that it closely resembles normal development activity. The package name, registry entry, and installation command may all appear legitimate, while the actual tarball and dependency graph have been manipulated by the attacker. Because there is no obvious prompt or manual review, victims may be compromised without noticing any abnormal behavior.

4.3 Limitations of Common Defenses

The Axios incident also reveals limitations in common dependency defenses. Tools such as `npm audit` and Snyk are valuable for known CVEs, but a newly published malicious version may not yet be listed in a vulnerability database. `npm ci` and lockfiles can improve reproducibility and verify

pinned versions and hashes from the registry; however, if a lockfile is generated during the malicious exposure window, it may preserve the compromised version. Dependabot can automatically update known vulnerable dependencies, but its decisions still heavily depend on CVEs and vulnerability feeds. An automated upgrade may also unintentionally introduce a newly compromised version.

Table 2: Common defenses and their limitations

Defense	Protection	Limitation
npm audit / Snyk	Detects known CVEs and known vulnerable dependencies in the dependency tree.	Reacts slowly to newly published malicious versions because the malicious version may not yet have a CVE or database entry.
npm ci + lockfile	Verifies pinned versions and registry hashes, improving build reproducibility.	Still trusts the lockfile; if the lockfile is generated during the attack window, it may lock in the malicious version.
Dependabot	Automatically updates known vulnerable dependencies.	Still depends on vulnerability databases; the update itself may introduce new risk.

Therefore, supply chain defense cannot rely only on known-vulnerability scanning. For malicious package publication, credential theft, and installation script abuse, defense must also consider package metadata, publishing behavior, lifecycle scripts, provenance status, and install-time risk checks.

5 The Core Gap: No Tamper-Evident Chain

The Axios incident reflects a more fundamental problem: between source code, build systems, registry packages, and final installation, traditional package installation workflows do not necessarily provide a verifiable tamper-evident chain. When developers install a package, they usually cannot clearly answer the following questions:

- Was the artifact built from the expected source commit?
- Were the build steps unmodified and not abused?
- Does the tarball published in the registry match the CI output?
- Was the artifact produced by the expected workflow, builder identity, and release process?

When these questions cannot be answered, attacks can remain silent. The package name, version number, and registry page may still appear trustworthy, but the artifact itself may not come from the expected source. The main value of build provenance is to create a signed, verifiable, and traceable evidence chain between source and installation.

6 Build Provenance as a Defense Mechanism

6.1 Concept of Build Provenance

Build provenance is metadata that describes how a software artifact was produced. It typically records the source repository, commit hash, builder identity, build workflow, build time, build type, and artifact hash. The presentation summarizes its meaning as: *Built from commit X, by pipeline Y, at time T*. If this record is signed and written to a transparency log, a consumer can check whether an artifact matches expectations during installation or verification.

In the SLSA and Sigstore model, build provenance is commonly combined with OIDC identity, keyless signing, artifact hashes, and the Rekor public append-only log. SLSA provides a supply chain integrity framework that explains what provenance should prove, while Sigstore provides signing and transparency log infrastructure that makes provenance statements more verifiable. The shared goal of these mechanisms is not merely to generate metadata, but to make the source-to-artifact path tamper-evident.

6.2 What Provenance Can Prove

Build provenance can help answer three core questions. First, it can record whether an artifact came from the expected source commit through a signed commit hash. Second, it can connect an artifact to a specific build

Table 3: Verification information provided by build provenance

Question	How Provenance Helps
Correct source commit?	The commit hash is recorded and signed, allowing the consumer to check whether the artifact came from the expected source revision.
Build steps unmodified?	The pipeline identity can be bound through OIDC and attestations, allowing the consumer to check whether the builder matches expectations.
Tarball matches CI output?	The artifact hash can be recorded in the provenance statement or transparency log, allowing the registry artifact to be compared with CI output.

pipeline through OIDC-bound builder identity. Third, it can check whether a registry tarball matches the attested CI output through an artifact hash.

Therefore, the core contribution of provenance is traceability. Consumers no longer need to rely only on the package name and version number; instead, they can require evidence about the origin, build process, and signature of the artifact.

6.3 Verification: Turning the Record into Prevention

Provenance itself is a record. To turn it into a preventive control, it must be verified at install time or during policy enforcement. The SLSA artifact verification model can be simplified into three steps. First, the consumer verifies the provenance signature to confirm that the attestation has not been tampered with. Second, the consumer compares the source repository, builder identity, and build type in the provenance against expected values. Third, in higher-assurance settings, the consumer may recursively verify dependencies.

Expected values may be declared by the maintainer in the repository, specified by organizational policy, or learned from the first observed attestation. This design allows provenance to become more than a passive record. It can function as an install-time gate. For example, if a previous version of a package had no attestation, a tool may issue a warning. If a previous version had an attestation but the target version suddenly lacks one, a tool may raise an error or block installation. If the target version has an attestation, the tool can verify the signature and continue the installation process.

6.4 Technology Stack and Workflow

In practice, build provenance involves several technology layers. SLSA defines what provenance and build levels are expected to prove. Sigstore provides keyless signing and the Rekor transparency log. Registries and CI platforms generate, store, and publish attestations. Consumer-side tools such as npq or Socket Firewall can serve as pre-install gates for provenance verification and can also help evaluate package installation risk.

Table 4: Technology layers related to build provenance

Layer	Technology	Role
Framework	SLSA	Defines what provenance should prove and describes build integrity levels.
Cryptography	Sigstore	Provides keyless signing and public transparency log support, including Rekor.
Platform	Registry + CI	Generates, stores, and publishes attestations for ecosystems such as npm, PyPI, or GitHub Actions.
Consumer	npq / Socket Firewall	Acts as a pre-install gate for provenance verification and helps evaluate package installation risk.

On the producer side, a CI/CD system typically builds an artifact from a specific source commit, generates a provenance statement, signs and records information such as the commit, builder, and artifact hash, and finally allows the registry to store the package together with its attestation metadata. On the consumer side, the package and its attestations are obtained before installation. The consumer then verifies the signature, checks the expected source, builder, and build type, and decides whether to warn, block, or allow installation according to policy.

6.5 Adoption Status

Provenance support has gradually appeared in major package ecosystems, but adoption is still at an early stage. The presentation material states that `npm publish --provenance` became generally available in October 2023; GitHub Artifact Attestations became generally available in June 2024; PyPI supported digital attestations in November 2024; and Maven, RubyGems, and Conda have also begun to support or experiment with related mechanisms.

Table 5: Adoption status of provenance-related mechanisms

Ecosystem	Status	Adoption	Source Men- tioned in Slides
npm	Generally available since October 2023.	3,800 packages during beta; more than 500M downloads of provenance-enabled packages.	GitHub Blog / Sigstore Blog
PyPI	Generally available since November 2024.	17% of uploads included attestations by the end of 2025.	PyPI Blog
GitHub Attestations	Generally available since June 2024.	No public adoption figure listed in the presentation.	GitHub Blog
Maven	Opt-in since January 2025.	Early adopters only.	Sonatype
RubyGems	In progress.	20 top gems were shipping attestations in March 2025.	RubyGems Blog
Conda	Beta since August 2025.	No public adoption figure listed in the presentation.	prefix.dev

This adoption status suggests that provenance is becoming more mature, but it has not yet become a universally enforced standard. The main limitations are not only whether tools support provenance, but also maintainer awareness, workflow migration overhead, and whether consumers actually verify provenance at install time rather than merely assuming safety because an attestation exists.

7 Case Study II: TanStack npm Supply-Chain Compromise

7.1 Incident Overview

The TanStack incident highlights the boundary of build provenance. According to the presentation material, TanStack suffered an npm supply chain compromise on May 11, 2026. In a short period of time, 42 packages and 84 malicious versions were published. The payload targeted cloud credentials, API tokens, and SSH keys. The incident was detected by an external researcher within 26 minutes.

The most important feature of this incident is that the malicious versions had valid build provenance. In other words, the problem was not that provenance was missing or that the attestation was completely unrelated to the package. Instead, provenance correctly recorded that the package was produced by TanStack's legitimate workflow. The real issue was that the cache and token trust boundary relied upon by the legitimate workflow had been abused by the attacker.

7.2 How the Attack Worked

The attack flow in the presentation can be summarized as follows: the attacker poisoned a shared CI cache through a fork pull request. When a later legitimate release workflow ran, it restored the poisoned cache. The restored cache then extracted an OIDC token from runner memory and published malicious packages through the legitimate release process. The simplified flow is shown below:

```
fork pull request
  -> shared CI cache poisoned
    -> legitimate release workflow restores cache
      -> OIDC token extracted from runner memory
        -> malicious packages published through legitimate
workflow
      -> valid build provenance generated
```

The key problem in this incident is a misplaced trust boundary. A fork pull request workflow belongs to a lower-trust context, while a release workflow belongs to a high-trust context. If the two workflows share caches or can influence each other, an attacker can use a low-trust entry point to

contaminate a high-trust release process. This shows that CI/CD caches, runner memory, OIDC tokens, and workflow permissions are attack surfaces that must be protected beyond provenance itself.

7.3 Why This Challenges Provenance

The TanStack incident demonstrates an important limitation: **provenance proves origin, not innocence**. Provenance can prove that an artifact came from a particular repository, workflow, or builder, but it cannot by itself prove that the workflow was clean when it executed. If a legitimate workflow is abused by an attacker, provenance will still accurately record that the artifact came from that workflow, even though the artifact may be malicious.

Therefore, build provenance is a necessary condition, not a sufficient condition. It can make the supply chain more transparent and traceable, and it can help consumers detect artifact substitution or missing attestations. However, it cannot replace CI/CD isolation, cache hardening, credential restriction, and workflow permission auditing. In other words, provenance raises the cost of attack, but an attacker may still bypass the guarantees provided by provenance by compromising the build environment.

7.4 Industry Response: OpenAI

The presentation also discusses OpenAI's response to the TanStack incident. During the TanStack incident, two OpenAI employee devices were affected while relevant controls were still in a phased rollout. Controls already deployed after the Axios incident included hardening credential materials in CI/CD pipelines, configuring package managers with `minimumReleaseAge` to reject newly published versions, and using security software to validate the provenance of new packages.

This response is consistent with the analysis above. Provenance verification is important, but organizations still need to reduce the probability that newly published malicious versions enter development environments. They also need to protect credentials stored or used in CI/CD systems and developer devices. These controls form a layered defense rather than relying on a single mechanism to solve all risks.

8 Toward a Layered Defense Strategy

The Axios and TanStack cases illustrate different dimensions of supply chain risk. Axios mainly demonstrates the problems of maintainer credentials, malicious package publication, and lifecycle script abuse. TanStack demonstrates the risks of CI/CD caches, OIDC tokens, and trusted workflow compromise. Together, the two cases show that no single defense mechanism can cover the entire supply chain attack surface. Practical defense must therefore be layered.

8.1 Dependency Layer

The goal of the dependency layer is to reduce the probability that malicious or vulnerable packages enter a project. In practice, organizations should use lockfiles to control versions and hashes, and they should avoid unnecessary dependency updates. SBOMs can record dependency composition, while vulnerability scanners such as `npm audit`, OSV, or Snyk can detect known vulnerabilities. However, these tools are insufficient for newly published malicious packages. Therefore, install-time gates such as `npq` or Socket Firewall should also be used to check package metadata, provenance status, and suspicious behavior. In high-risk environments, a minimum release age policy can reduce the probability that a newly published malicious version enters a project immediately.

8.2 Source Layer

The source layer protects source repositories and maintainer accounts. If a maintainer account is stolen, the attacker may directly publish malicious versions or modify the release process. Therefore, multi-factor authentication, protected branches, mandatory code review, and restricted publishing permissions are necessary controls. Organizations should also monitor abnormal commits, releases, token usage, and maintainer activity so that suspicious behavior from trusted accounts can be detected early.

8.3 Build Layer

The build layer is the most important lesson from the TanStack incident. Organizations must strictly separate untrusted pull request workflows from trusted release workflows, and should avoid sharing high-trust caches be-

tween fork pull request workflows and release workflows. GitHub Actions tokens, OIDC tokens, and registry tokens should follow the principle of least privilege, and credential materials should not appear in runner memory, logs, or locations readable by low-trust processes. Cache restore paths, build scripts, and release scripts should be treated as part of the security boundary, not merely as performance optimization tools.

8.4 Release Layer

The goal of the release layer is to make artifacts verifiable during publication and installation. Organizations should use signed attestations and build provenance to record source commits, builder identities, artifact hashes, and build workflows in verifiable metadata. More importantly, consumers should verify provenance at install time rather than merely generating provenance at publish time. If a package previously had an attestation but a new version suddenly lacks one, installation tools should at least warn the user, and may block installation according to policy.

9 Discussion

9.1 Practical Value of Build Provenance

The practical value of build provenance is that it converts implicit trust relationships into verifiable evidence. In traditional package ecosystems, consumers often see only the package name, version number, and registry metadata, while having little ability to confirm that the artifact actually came from the expected source code and build process. Provenance establishes a relationship among source, builder, and artifact through signed attestations, making artifact substitution, unauthorized publishing, and some maintainer-compromise scenarios easier to detect.

In addition, provenance provides a basis for policy enforcement. An enterprise or organization can require installed packages to come from specific repositories, specific builders, or valid attestations. This shifts supply chain security from “assuming that the package should be correct” to “requiring the package to provide machine-verifiable evidence.” From this perspective, provenance is not merely a logging mechanism, but also infrastructure that can be used by automated security policies.

9.2 Fundamental Limitation of Build Provenance

However, provenance should not be confused with a malware scanner, nor should it be treated as a complete proof of build environment integrity. The TanStack incident shows that if a legitimate workflow is contaminated, provenance may still be completely valid. What it proves is the origin of the artifact, not the benign nature of the artifact. In other words, provenance can answer “where did this artifact come from,” but it cannot by itself answer “is this artifact safe?”

A more precise conclusion is therefore: **build provenance raises the bar, but the bar can still be cleared**. It makes it more difficult for an attacker to forge artifact origin and makes missing provenance or unexpected builders easier to detect. However, an attacker may still bypass part of this guarantee through CI/CD cache poisoning, credential theft, workflow abuse, or maintainer account compromise. Provenance should be adopted and verified, but it must be used together with CI/CD isolation, cache hardening, credential restriction, dependency scanning, and install-time controls.

9.3 Synthesis of the Two Case Studies

The difference between Axios and TanStack helps clarify the full scope of supply chain security. The Axios incident is closer to package publication and lifecycle script abuse: the attacker used stolen maintainer credentials to publish a malicious package version and completed compromise through an installation script. In this scenario, provenance and install-time verification can increase the chance that users detect abnormal conditions before installation.

The TanStack incident is closer to build pipeline compromise: the attacker did not simply forge the origin of the artifact, but caused a legitimate workflow to produce a malicious artifact. In this scenario, provenance may still produce an apparently correct attestation. Therefore, the TanStack incident reminds us that the trustworthiness of build provenance depends on the trustworthiness of the build environment. If the trust boundary of the build environment is poorly designed, provenance can only faithfully record a process that has already been contaminated.

10 Conclusion

Software supply chain attacks are dangerous because they exploit the trust structure behind the efficiency of modern software development. Developers trust package managers, package maintainers, registries, CI/CD workflows, build caches, and automated release processes. Attackers target these trusted nodes so that malicious code can enter downstream systems while preserving the appearance of normal development activity.

The Axios incident shows that stolen maintainer credentials, malicious package publication, hidden dependencies, and `postinstall` scripts can compromise users through a normal installation process. The TanStack incident further shows that even when malicious packages have valid build provenance, provenance may still be insufficient if a CI/CD workflow or shared cache has already been compromised. Together, the two cases support the central argument of this report: build provenance is necessary, but not sufficient.

The real value of build provenance is that it makes the relationship among source, builder, and artifact more visible, signable, and verifiable. It can support install-time verification and policy enforcement, and it can increase the visibility of artifact substitution or missing attestations. However, it cannot replace a secure build environment, nor can it independently prove that an artifact is benign. Future software supply chain defense should therefore treat provenance as a foundational mechanism while also deploying dependency scanning, lockfile discipline, minimum release age policies, maintainer account protection, CI/CD isolation, cache hardening, credential restriction, and provenance verification. Only through this layered defense can organizations reasonably reduce the systemic risk of modern software supply chain attacks.

References

- [1] National Institute of Standards and Technology. *NIST SP 800-204D: Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204D.pdf>
- [2] OWASP. *Software Supply Chain Security Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Software_Supply_Chain_Security_Cheat_Sheet.html
- [3] OWASP. *A03:2025 Software Supply Chain Failures*. https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/
- [4] OWASP. *OWASP Top 10:2025 Introduction and Methodology*. https://owasp.org/Top10/2025/0x00_2025-Introduction/
- [5] SLSA. *Supply Chain Threats*. <https://slsa.dev/spec/v1.0/threats-overview>
- [6] SLSA. *About SLSA*. <https://slsa.dev/spec/v1.2/about>
- [7] SLSA. *Verifying Artifacts*. <https://slsa.dev/spec/v1.0/verifying-artifacts>
- [8] OpenSSF. *Sigstore*. <https://www.sigstore.dev/>
- [9] Liran Tal. *npq: Safely install packages with npm or yarn by auditing them as part of your install process*. <https://github.com/lirantal/npq>
- [10] Socket. *Introducing Socket Firewall*. <https://socket.dev/blog/introducing-socket-firewall>
- [11] Axios. *Post Mortem: axios npm supply chain compromise*. <https://github.com/axios/axios/issues/10636>
- [12] Microsoft Security Blog. *Mitigating the Axios npm supply chain compromise*. <https://www.microsoft.com/en-us/security/blog/2026/04/01/mitigating-the-axios-npm-supply-chain-compromise/>
- [13] OpenAI. *Our response to the Axios developer tool compromise*. <https://openai.com/index/axios-developer-tool-compromise>

- [14] TanStack. *Postmortem: TanStack npm supply-chain compromise.* <https://tanstack.com/blog/npm-supply-chain-compromise-postmortem>
- [15] TanStack. *Hardening TanStack After the npm Compromise.* <https://tanstack.com/blog/incident-followup>
- [16] OpenAI. *Our response to the TanStack npm supply chain attack.* <https://openai.com/index/our-response-to-the-tanstack-npm-supply-chain-attack>
- [17] GitHub Blog. *Artifact Attestations is generally available.* <https://github.blog/changelog/2024-06-25-artifact-attestations-is-generally-available/>
- [18] GitHub Blog. *Introducing npm package provenance.* <https://github.blog/security/supply-chain-security/introducing-npm-package-provenance/>
- [19] Sigstore Blog. *npm's Sigstore-powered provenance goes GA.* <https://blog.sigstore.dev/npm-provenance-ga/>
- [20] PyPI Blog. *PyPI now supports digital attestations.* <https://blog.pypi.org/posts/2024-11-14-pypi-now-supports-digital-attestations/>
- [21] PyPI Blog. *PyPI 2025 Year in Review.* <https://blog.pypi.org/posts/2025-12-31-pypi-2025-in-review/>
- [22] Sonatype Central. *Sigstore Signature Validation via Portal.* https://central.sonatype.org/news/20250128_sigstore_signature_validation_via_portal/
- [23] RubyGems Blog. *February 2025 Updates.* <https://blog.rubygems.org/2025/03/19/february-rubygems-updates.html>
- [24] prefix.dev. *Securing the Conda package supply chain with Sigstore.* <https://prefix.dev/blog/securing-the-conda-package-supply-chain-with-sigstore>
- [25] IBM. *Cost of a Data Breach Report 2025 / Attack vector overview.* <https://www.ibm.com/think/topics/attack-vector>

- [26] ReversingLabs. *Software Supply Chain Security Report 2026*. <https://www.reversinglabs.com/resources/software-supply-chain-security-report-2026>